

Software Architecture

Exercise – Architecture Patterns (From Mud To Structure)

BSc



Ingo Arnold

Copyright Notice

© Ingo Arnold. All rights reserved.

You may use these materials for your personal internal reference purposes only.

You may not reuse these materials in creating your own educational offerings or in your own educational situations like courses, lectures, or seminars nor may you distribute, transfer, resell or otherwise make them available to any other person or party without the expressed permission of the author.

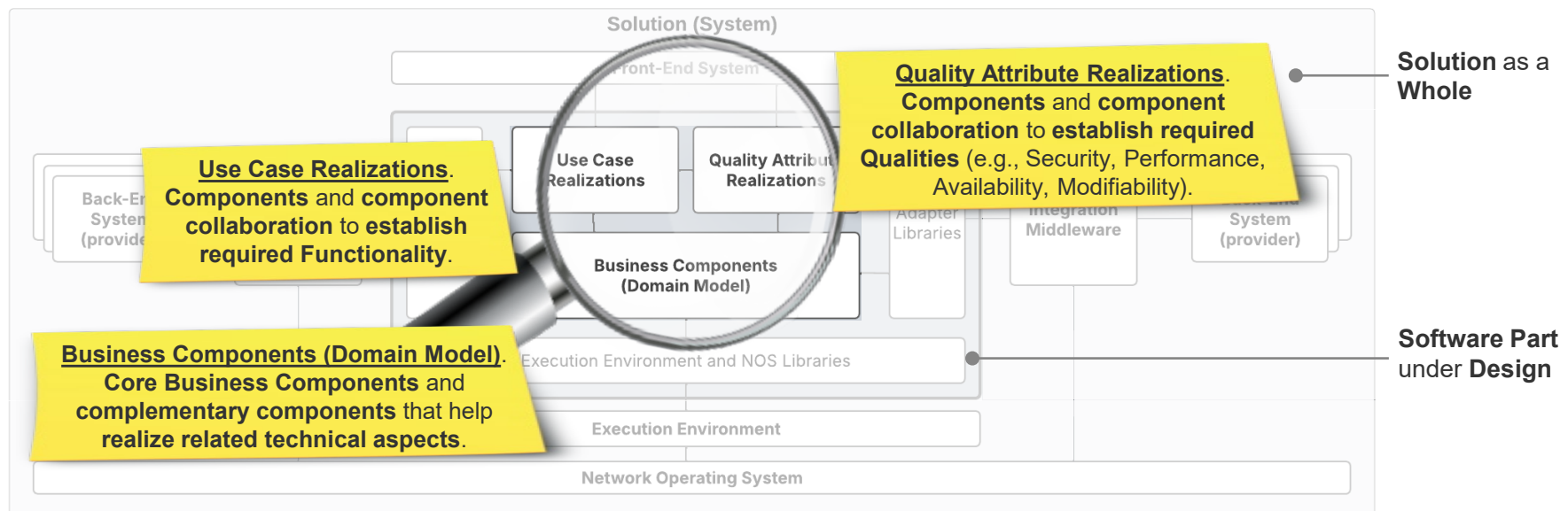
URLs or other references to Internet Web sites in the training materials are subject to change without notice. Unless otherwise indicated, all companies, organisations, domain names, email addresses, persons, places and events depicted in the Training Materials are for illustrative purposes only and are fictitious. No real connection is intended or inferred.

These training materials are provided “as is” and the author makes no warranties, express or implied, with respect to these training materials.

Exercise Opening

Architecture Context

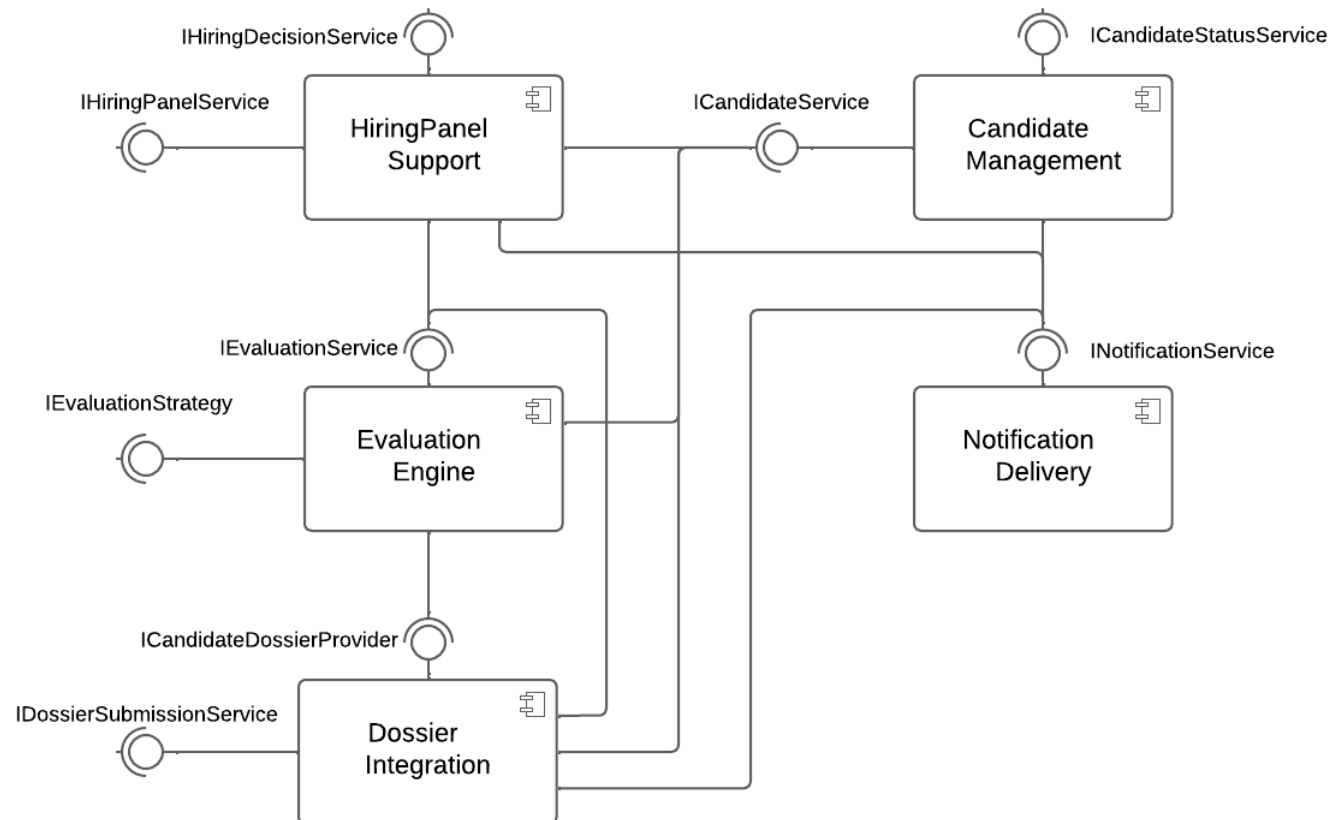
In today's exercise, we will look at the **adaption of Architecture Patterns** to address **core Domain Design** as well as **Functional** and **Non-Functional Requirements**.



Exercise Opening

Overview

In today's **exercise**, we **continue** what we **started last week** (Component Orientation), where we **arrived at the following**.



Exercise Opening

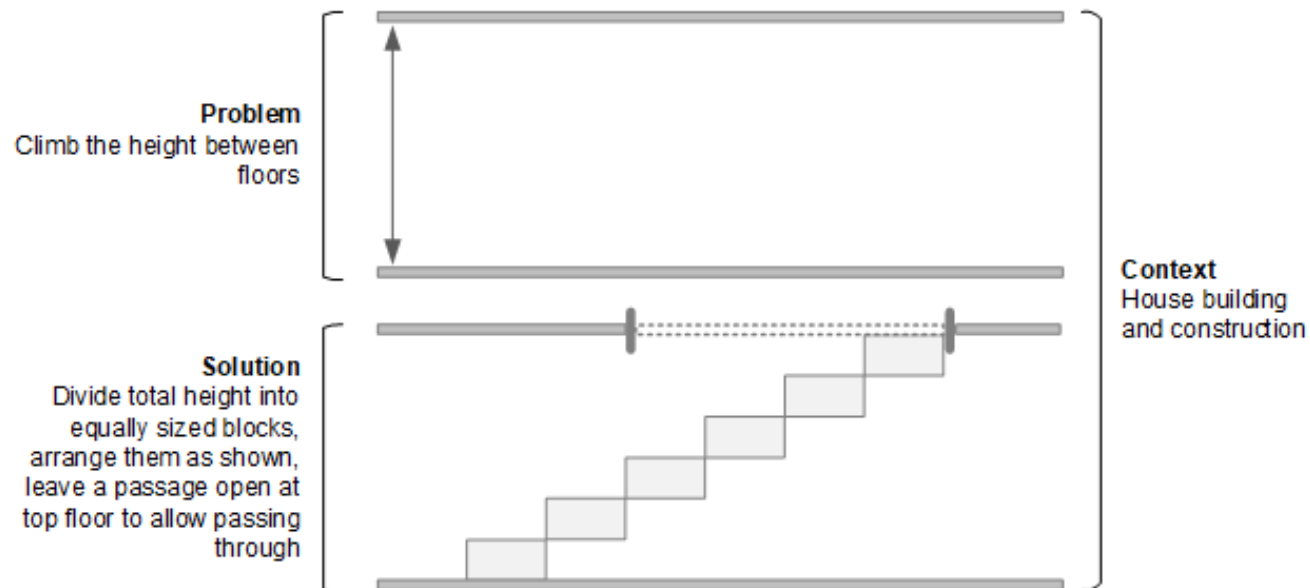
Overview

We want to **further improve** our **design** by **adapting Architecture Mud to Structure Patterns** wherever meaningful to achieve further features or quality aspects.

Architecture Patterns

Repeating Core Concepts

From a problem-solving perspective, each **pattern** is a **recipe** that describes what one must do to create the design that the pattern prescribes.

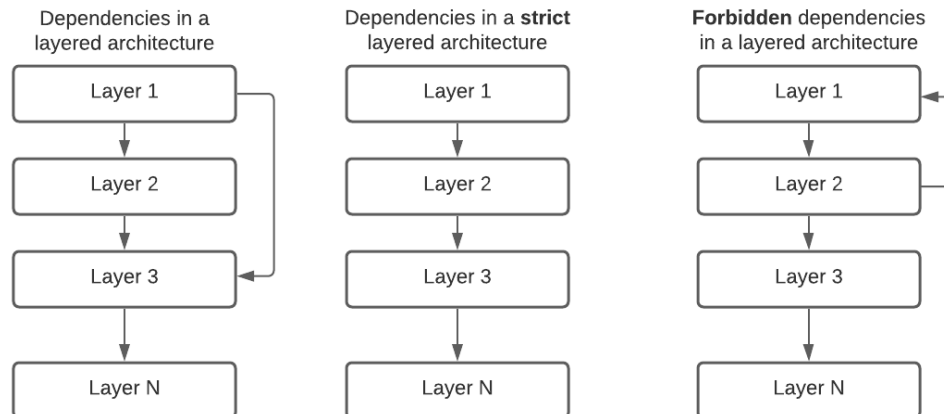


Architecture Patterns

Repeating Core Concepts – Layers

Layers Pattern

- **Problem:** Complex systems tend to become entangled when all parts interact freely. This makes maintenance, reuse, and substitution of functionality difficult.
- **Key building blocks**
 - Layers (e.g., Presentation, Business Logic, Data Access)
 - Interfaces between layers
- **Solution:** Each layer provides services to the layer above it and uses services from the layer below it. This enforces clear dependency direction, hides internal details, and allows independent evolution of layers.

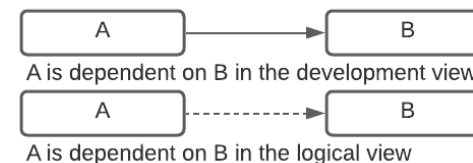
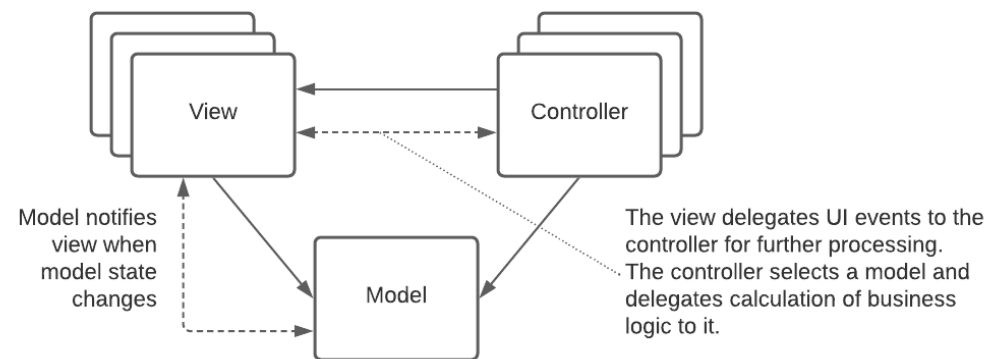


Architecture Patterns

Repeating Core Concepts – Model-View-Controller

Model-View-Controller (MVC) Pattern

- **Problem:** User interface code often becomes tightly coupled with business logic, making systems hard to adapt to different UIs or devices.
- **Key building blocks**
 - Model: domain data and business logic
 - View: visual representation of the Model's data
 - Controller: mediator handling user input and coordinating updates
- **Solution:** The Controller interprets user actions and updates the Model. The Model notifies Views of changes (often via the Observer pattern). The Views refresh automatically. This separation allows independent modification of UI and logic.

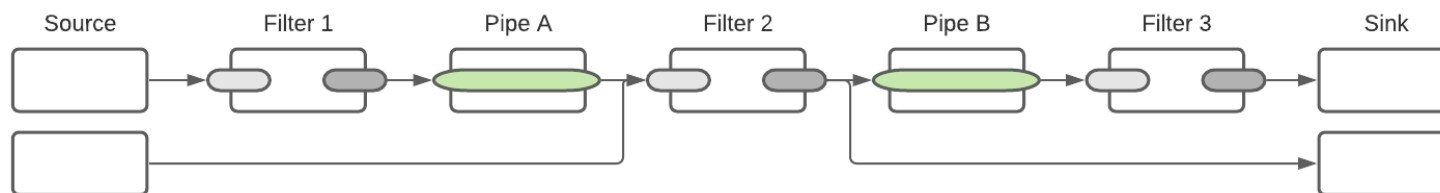


Architecture Patterns

Repeating Core Concepts – Pipes and Filters

Pipes and Filters Pattern

- **Problem:** Applications that perform complex data transformations often mix transformation logic and control flow, reducing flexibility and reuse.
- **Key building blocks**
 - Filters: independent processing units transforming input to output
 - Pipes: connectors that pass data streams between filters
- **Solution:** Each filter processes data received from an upstream pipe and passes its output downstream. Filters can be rearranged or replaced to form new processing pipelines without changing the overall control structure.

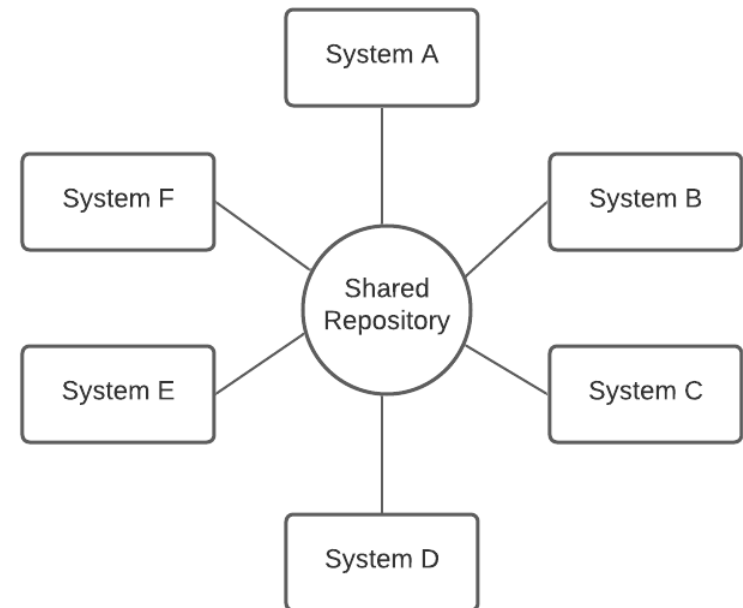


Architecture Patterns

Repeating Core Concepts – Shared Repository

Shared Repository Pattern

- **Problem:** Subsystems that need to share and maintain large amounts of common data often build their own ad-hoc data exchange logic, leading to redundancy and inconsistency.
- **Key building blocks**
 - Repository (central data store)
 - Subsystems or components accessing the repository via a defined schema or interface
- **Solution:** All components read and write to a common repository through standardized access interfaces. Consistency is ensured centrally, and each component can focus on its own processing logic rather than data exchange.

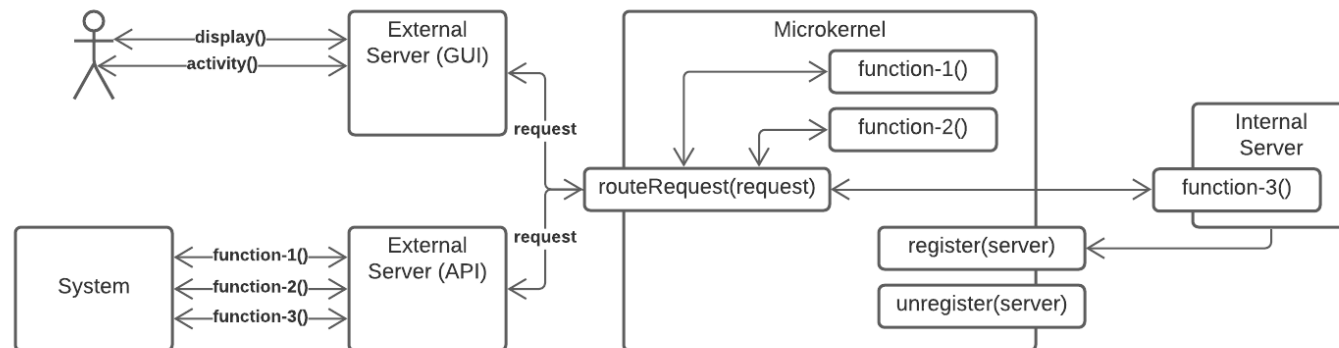


Architecture Patterns

Repeating Core Concepts – Microkernel

Microkernel Pattern

- **Problem:** Systems with a stable core but many variable extensions (e.g., operating systems or domain frameworks) need a way to evolve without modifying the core.
- **Key building blocks**
 - Microkernel (core services): minimal runtime, communication, and management
 - Internal servers: core subsystems directly managed by the kernel
 - External plug-ins/adapters: application-specific extensions
- **Solution:** The microkernel provides basic mechanisms (e.g., service registration, communication). Plug-ins extend functionality through well-defined extension points. This enables portability, scalability, and flexibility while keeping the kernel stable.

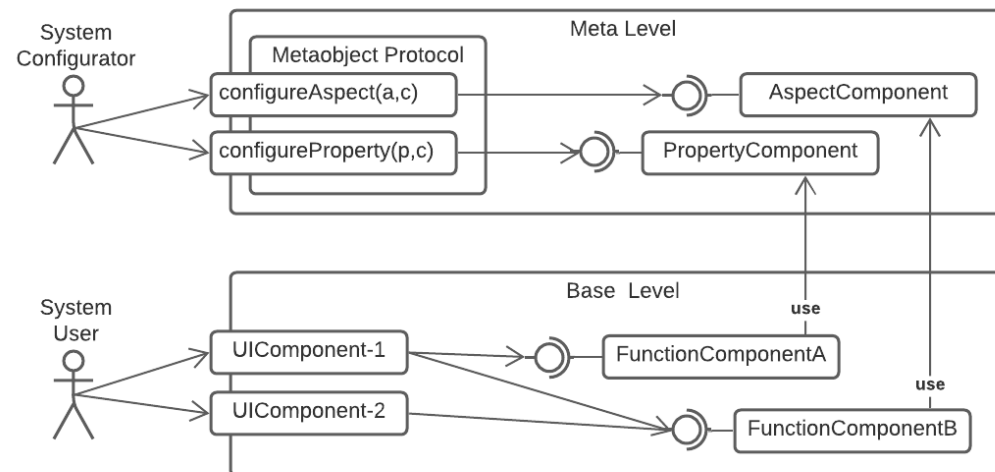


Architecture Patterns

Repeating Core Concepts – Reflection

Reflection Pattern

- **Problem:** Highly dynamic systems must adapt behavior or structure at runtime without recompilation, but typical static designs hard-code such logic.
- **Key building blocks**
 - Meta-level + its components: maintains representations of system structure and behavior (metadata)
 - Base-level + its components: performs normal application logic using meta-level definitions
- **Solution:** The base-level consults or modifies its meta-level descriptions at runtime to adapt behavior dynamically (e.g., changing object bindings or rules). This allows self-inspection, self-adaptation, and runtime reconfiguration.



Case Study

BPO Example

We will **divide** into **5 groups** this time.

Each group will aim to **improve** the **structure, maintainability, or extensibility** of the **CandidateInsight application design** developed to date **adapting** one of the following **5 Architecture Patterns**:

- Layers
- Model-View-Controller
- Pipes and Filters
- Shared Repository
- Microkernel

Case Study

BPO Example

List of **Architecture Patterns** and **Student Groups**.

Architecture Pattern	General Goal	CandidateInsight-Specific Goal	Working Group
Layers	Clear layer formation and separation of responsibilities.	How can you divide CandidateManagement, EvaluationEngine, and NotificationDelivery into clean layers (e.g., Application / Domain / Infrastructure)? What rules apply to dependencies? Consider a small example (e.g., the use case “Candidate evaluation completed and send notification to candidate”) to sketch a respective solution design.	Group 1
Model-View-Controller	Clean separation of presentation, logic, and data.	How could HiringPanelSupport be modelled as an MVC structure (e.g., web interface = View, Panel Controller = Controller, CandidateProfile = Model)? Consider a small example (e.g., the use case “Panel member approves a candidate”) and outline a corresponding solution design.	Group 2
Pipes and Filters	Structured, expandable data processing.	How could you design the data flow in the EvaluationEngine (e.g., input → normalization → scoring → persistence → notification) as a filter chain? Use the “Automated Pipeline for Applicant Evaluation” use case as an example and show how the evaluation of a new candidate could be propagated through the above filters.	Group 3

Case Study

BPO Example

List of **Architecture Patterns** and **Student Groups**.

Architecture Pattern	General Goal	CandidateInsight-Specific Goal	Working Group
Shared Repository	Central data and status management.	How can CandidateManagement provide a shared repository that is used by EvaluationEngine, HiringPanelSupport, and DossierIntegration without creating tight coupling? Consider using a shared repository for the use case “Evaluation results are stored and later reviewed by the hiring committee,” in which the EvaluationEngine component writes an evaluation to the repository and the HiringPanelSupport component reads the evaluation from the repository later.	Group 4
Microkernel	Expandability through plug-ins or modular services.	How could the EvaluationEngine be converted into a microkernel in which new evaluation strategies are registered as plug-ins? How do plug-ins communicate with the kernel? For example, consider a use case “Pluggable Evaluation Strategies for Different Job Profiles”, for which the Evaluation Engine should be able to evaluate candidates using different strategies (e.g., rule-based strategy for standard positions, ML-based strategy for data-intensive profiles). At the same time, it should be possible to register and use new strategies without changing the code of the core system.	Group 5

Case Study

BPO Example

Common guiding questions for all groups:

1. **What** is the **basic principle** of **your pattern**?
 - Explain it briefly in your own words.
2. **Which problem** in the **CandidateInsight design** does **your pattern** address?
 - Which component(s) benefit the most?
3. **How** do you **apply** the **pattern** in **practice**?
 - Show an adapted architecture diagram or a UML sketch.
 - Describe how components or interfaces are changed.
4. **What improvements result**?
 - Name the quality attributes affected (e.g., coupling, cohesion, extensibility, testability). What risks or costs arise?
5. **How** could **your pattern** be **combined** with **others**?



example

Case Study

BPO Example – Layers

Here I will share my reference solutions with you for the following Architecture Patterns:

- Layers
- Model-View-Controller
- Pipes and Filters
- Shared Repository
- Microkernel

Questions

